CITS2200: Data Structures and Algorithms

Lecture 2: Referential arrays in Python

Referential Array

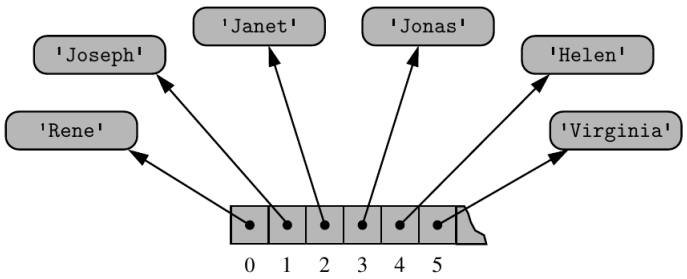

Figure 5.4: An array storing references to strings.

Figure 1: ©M. Goodrich : Data Structures and Algorithms in Python

- Consider storing strings of variable lengths in an array (the picture above).
- Now each location of the array stores the *reference* of the string.
- The actual strings are stored in other parts of memory, each is a string object.
- A reference is simply an address or the number of the first byte of the string.
- Accessing a string now requires two lookups. For example, if this array is A, $A[2]$ will store the number of the first byte of the location (or address of the location) where the string `Janet` is stored.
- Though the programmer has a mental picture that $A[2]$ stores the string `Janet`.

`list` class

- Lists are the most important data structure provided in Python, and we will make extensive use of lists in this unit.
- Lists in Python are implemented using arrays.
- `['red', 'green', 'blue', 'pink']` is a list. A list holds elements that can be iterated upon (we will discuss `iterator`s soon).
- Note the `[]` to indicate a list. If there is nothing inside `[]`, it indicates an empty list.
- But we can initialize a list
```
colors = ['red', 'green', 'blue', 'pink']
primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31]
```

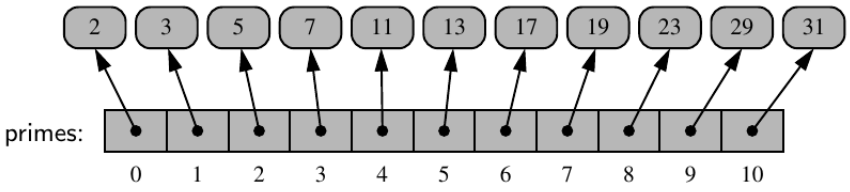

Figure 2: ©M. Goodrich : Data Structures and Algorithms in Python

Lists store references to objects

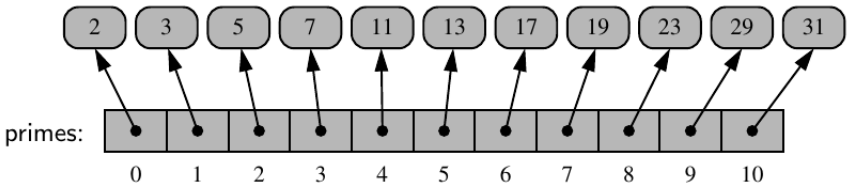

Figure 3: ©M. Goodrich : Data Structures and Algorithms in Python

- Note this figure carefully. The list does not store the elements or members of the list.
- That is because the list may have members of different sizes. Though each member is an integer for `primes`, each member is a string of different length for `colors`.
- But we should be able to access any member of a list quickly. So it is more efficient to keep references to the members in the array, and keep the actual members elsewhere in memory.
- A *reference* is an *address* or the number of the starting byte of the member (where it is stored in memory).
- It is important to keep this in mind, though we pretend that the list itself stores the members, for example, `primes[0]` returns the integer 2.

Compact arrays

- Strings are stored as *compact arrays* and not as referential arrays in Python.
- This means the characters in a string are actually stored in the array, and not their references.

Figure 4: ©M. Goodrich : Data Structures and Algorithms in Python

- This is possible because each character can be represented as unicode using two bytes. And Python knows this in advance.
- We will learn about *time* and *space* complexities for solving problems. Time complexity is the time it takes to run a program (an algorithm) and space complexity is the memory space our solution takes.
- Referential arrays are not good in terms of space complexity. They have to store a reference to an object, and also the actual object somewhere in memory.
- But we will mostly use referential structures in this unit, as Python implements lists that way.

`dict` class

- The `dict` or *dictionary* class is one of the most important classes for us.
- A dictionary is a special kind of list. Each member of a list is a single element, each member of a dictionary is a *pair* of elements called a *mapping*.
- For example, the dictionary {`'ga':'Irish', 'de':'German'` } is a dictionary.
- It maps `'ge'` to `'Irish'` and `'de'` to `'German'`.
- ```
  pairs = [('ge','Irish'), ('de','German')]
  dict(pairs)
  ```
  ```

Dynamic arrays

- Compact arrays are used for *immutable* (data that does not change over time) data like string.
- But a more flexible allocation is when an array can be changed (the size can be increased) over time.
- If the required size is not known when the array is allocated first time, we have two choices.
- We can either allocate a very large array, but this may waste memory space if we do not need that large an array.
- The other alternative is to 'grow' the array if it is full.
- But the difficulty is, there may not be any space available in the memory next to the array, as that space may have been allocated to other objects or arrays.
- So a larger space is allocated in another part of the memory, and the current array is copied to that larger space.

Dynamic arrays

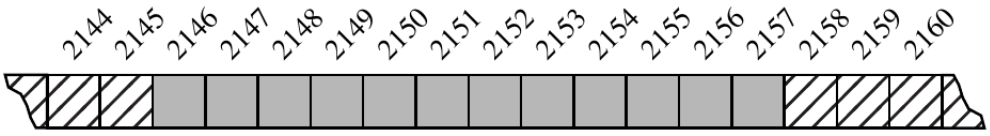

Figure 5.11: An array of 12 bytes allocated in memory locations 2146 through 2157.

Figure 5: ©M. Goodrich : Data Structures and Algorithms in Python

- Python's `list` class provides this interesting abstraction.
- We have to get used to the word *abstraction*. In simple terms it means how we use some data structure may be different from how it is implemented in Python.
- For example, we can go on adding new elements to a list without worrying how Python is implementing the list.
- The first abstraction is, the actual array for a *list* instance may be larger than what the user allocates.
- This is done to keep the flexibility so that new elements can be added when the list is full from the programmer's point of view.
- However, when the list is actually full eventually, Python requests the system for a larger block of memory. The old array is copied to the beginning of this array, and the programmer can now continue adding elements to the list.
- Note that you will not notice any of this as a programmer (you work with an abstraction of the list). We will analyse the complexity of dynamic arrays when we discuss *amortized analysis*.

Dynamic arrays

```
import sys
 data = []

 for k in range(10):
  a= len(data)
  b= sys.getsizeof(data)
  print('Length: {0:3d}; Size in bytes: {1:4d}'.format(a,b))
  data.append(None)
```

- Here we take an empty array and append elements.
- The elements are `None` which is a null value or no value, a special data type in Python. It is not 0, `false` or an empty string, it is `None`.
- `import sys` is required for using the `getsizeof` function.

```
Length:   0; Size in bytes:   56
Length:   1; Size in bytes:   88
Length:   2; Size in bytes:   88
Length:   3; Size in bytes:   88
Length:   4; Size in bytes:   88
Length:   5; Size in bytes:  120
Length:   6; Size in bytes:  120
Length:   7; Size in bytes:  120
Length:   8; Size in bytes:  120
Length:   9; Size in bytes:  184
```

- Python starts by allocating an array of size 56 bytes. The array size is increased as more elements are added.

Dynamic arrays

- Almost all computers now are 64-bit. This means, the address of each memory location is represented as a 64 bit binary number. We will not discuss binary numbers here. 64 bits = 8 bytes.
- Any increase in size of the array should be in multiples of 8 as we need at least 8 bytes to store a single reference (we are using a referential array).
- Python first allocates 56 bytes ($7 \times 8 = 56$) that can store 7 object references. Why? We don't know. It may be a policy of local installation of Python.
- The array size was increased to 88 bytes ($11 \times 8 = 88$) to store 11 object references, as soon as we appended one element. Perhaps Python expected us to add more elements.
- The size was increased to 120 bytes ($15 \times 8 = 120$) to store 15 object references once we added 4 more elements.
- You can check the storage allocation for many more elements.

Dynamic arrays

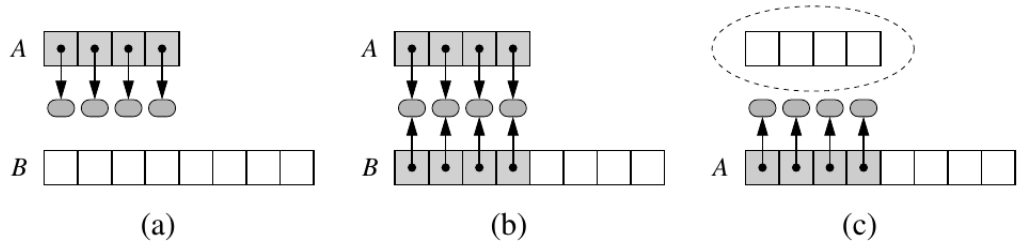

Figure 5.12: An illustration of the three steps for "growing" a dynamic array: (a) create new array B; (b) store elements of A in B; (c) reassign reference A to the new array. Not shown is the future garbage collection of the old array, or the insertion of the new element.

Figure 6: ©M. Goodrich : Data Structures and Algorithms in Python

Dynamic arrays

```
1  import ctypes                                    # provides low-level arrays
2
3  class DynamicArray:
4    """A dynamic array class akin to a simplified Python list."""
5
6    def __init__(self):
7      """Create an empty array."""
8      self._n = 0                                  # count actual elements
9      self._capacity = 1                           # default array capacity
10     self._A = self._make_array(self._capacity)   # low-level array
11
12   def __len__(self):
13     """Return number of elements stored in the array."""
14     return self._n
```

Figure 7: ©M. Goodrich : Data Structures and Algorithms in Python

- Some common coding conventions are used. Encapsulated variables (or class variables that cannot be accessed from outside the class without invoking a method of the class) start with an underscore `'_'`.
- Class methods start with two leading underscores and are followed by two trailing underscores.
- `self` is a reference for the current instance of a class. We create many instances of a class in a program, but use `self` to refer to the current instance.
- `ctype` provides facilities for creation of low level arrays. We will not discuss this module as we will mostly use referential arrays. `ctype` comes from the C language. Python interpreter is written in the C language.

```
16    def __getitem__(self, k):
17      """Return element at index k."""
18      if not 0 <= k < self._n:
19        raise IndexError('invalid index')
20      return self._A[k]                          # retrieve from array
21
22    def append(self, obj):
23      """Add object to end of the array."""
24      if self._n == self._capacity:              # not enough room
25        self._resize(2 * self._capacity)         # so double capacity
26      self._A[self._n] = obj
27      self._n += 1
28
```

Figure 8: ©M. Goodrich : Data Structures and Algorithms in Python

- `__getitem__` returns an item if it exists.
- `append` tries to append an object to the array if it still has capacity, otherwise it calls `__resize__` to double the size of the array.

```
29    def _resize(self, c):                    # nonpublic utitity
30      """Resize internal array to capacity c."""
31      B = self._make_array(c)                # new (bigger) array
32      for k in range(self._n):               # for each existing value
33        B[k] = self._A[k]
34      self._A = B                            # use the bigger array
35      self._capacity = c
36
37    def _make_array(self, c):                # nonpublic utitity
38      """Return new array with capacity c."""
39      return (c * ctypes.py_object)( )       # see ctypes documentation
```

Code Fragment 5.3: An implementation of a DynamicArray class, using a raw array from the ctypes module as storage.

Figure 9: ©M. Goodrich : Data Structures and Algorithms in Python

- `_resize` allocates a new array and copies the existing elements at the start of the new array.